

Лабораторная работа №4

Егор Мальцев

Z33434

Цель

Познакомить студента с принципами параллельных вычислений. Составить несколько программ в простейшими вычислительными действиями, чтобы освоить принципы параллельных вычислений (когда одни алгоритмы зависят / не зависят от других).

Задачи

1. [C++ SEQUENCE] Последовательные вычисления

Требуется последовательно выполнить вычисления по формуле 1, вычисления по формуле 2, после чего выполнить вычисления по формуле 3, которые выглядят следующим образом: результат вычислений 1 + результат вычислений 2 – результат вычислений 1

Выполнить последовательно на 10 000 итераций и 100 000 итераций

Формула 1: $f(x) = x^2 - x^2 + x^4 - x^5 + x + x$

Формула 2: $f(x) = x + x$

Вывести длительность выполнения всех 10 000 итераций и 100 000 итераций в сек.

2. [C++ THREADS] Параллельные вычисления через потоки

Требуется параллельно (насколько возможно с помощью потоков) выполнить вычисления по формуле 1, вычисления по формуле 2, после чего выполнить вычисления по формуле 3, которые выглядят следующим образом: результат вычислений 1 + результат вычислений 2 – результат вычислений 1

Выполнить последовательно на 10 000 итераций и 100 000 итераций

Формула 1: $f(x) = x^2 - x^2 + x^4 - x^5 + x + x$

Формула 2: $f(x) = x + x$

3. [C++ PROCESS] Параллельные вычисления через процессы

Требуется параллельно (насколько возможно с помощью процессов) выполнить вычисления по формуле 1, вычисления по формуле 2, после чего выполнить вычисления по формуле 3, которые выглядят следующим образом: результат вычислений 1 + результат вычислений 2 – результат вычислений 1

Выполнить последовательно на 10 000 итераций и 100 000 итераций

Формула 1: $f(x) = x^2 - x^2 + x^4 - x^5 + x + x$

Формула 2: $f(x) = x + x$

3. [SAVE] Результат всех вышеперечисленных шагов сохранить в репозиторий (+ отчет по данной ЛР в папку doc)

Фиксацию ревизий производить строго через ветку dev. С помощью скриптов накатить ревизии на stg и на prd. Скрипты разместить в корне репозитория. Также создать скрипты по возврату к виду текущей ревизии (даже если в папке имеются несохраненные изменения + новые файлы).

Выполнение

SEQUENCE

```
1 #include <iostream>
2 #include <string>
3 #include <cmath>
4 #include <chrono>
5
6 double F1(double x) {
7     return x * x - x * x + x * 4 - x * 5 + x + x;
8 }
9
10 double F2(double x) {
11     return x + x;
12 }
13
14 double F3(double x, double y) {
15     return x + y - x;
16 }
17
18 double Full(double x) {
19     double a, b;
20     a = F1(x);
21     b = F2(x);
22     return F3(a, b);
23 }
24
25 int main() {
26     double x1, x2;
27     int n1 = 10000;
28     int n2 = 100000;
29     std::cin >> x1;
30     x2 = x1;
31
32     auto start = std::chrono::high_resolution_clock::now();
33     for (int i = 0; i != n1; ++i) {
34         x1 = Full(x1);
35     }
36     auto end = std::chrono::high_resolution_clock::now();
37     auto diff = end - start;
38
39     std::cout << std::chrono::duration<double, std::chrono::seconds::period>(diff).count() << " s\n";
40
41     auto start2 = std::chrono::high_resolution_clock::now();
42     for (int i = 0; i != n2; ++i) {
43         x2 = Full(x2);
44     }
45     auto end2 = std::chrono::high_resolution_clock::now();
46     auto diff2 = end2 - start2;
47     std::cout << std::chrono::duration<double, std::chrono::seconds::period>(diff2).count() << " s\n";
48 }
```

```
root@DESKTOP-7RFVCBH:/usr/local/labs/lab_04# ./sequence_diff
3.14
0.000214702 s
0.00177799 s
```

THREADS

```
1 #include <iostream>
2 #include <string>
3 #include <cmath>
4 #include <chrono>
5 #include <thread>
6
7 double a, b;
8
9 void F1(double x)
10 {
11     a = pow(x, 2) - pow(x, 2) + x * 4 - x * 5 + x + x;
12 }
13
14 void F2(double x)
15 {
16     b = x + x;
17 }
18
19 double F3(double x, double y) {
20     return x + y - x;
21 }
22
23 double Full(double x) {
24     std::thread at(F1, x);
25     std::thread bt(F2, x);
26     at.join();
27     bt.join();
28     return F3(a, b);
29 }
30
31 int main() {
32     double x1, x2;
33     int n1 = 10000;
34     int n2 = 100000;
35     std::cin >> x1;
36     x2 = x1;
37     auto start = std::chrono::high_resolution_clock::now();
38     for (int i = 0; i != n1; ++i) {
39         x1 = Full(x1);
40     }
41     auto end = std::chrono::high_resolution_clock::now();
42     auto diff = end - start;
43
44     std::cout << std::chrono::duration<double, std::chrono::seconds::period>(diff).count() << " s\n";
45
46     auto start2 = std::chrono::high_resolution_clock::now();
47     for (int i = 0; i != n2; ++i) {
48         x2 = Full(x2);
49     }
50     auto end2 = std::chrono::high_resolution_clock::now();
51     auto diff2 = end2 - start2;
52     std::cout << std::chrono::duration<double, std::chrono::seconds::period>(diff2).count() << " s\n";
53 }
```

```
root@DESKTOP-7RFVCBH:/usr/local/labs/lab_04# ./threads_diff
3.14
0.854454 s
8.21295 s
```

Заключение

1. Эффективнее всего оказался последовательный алгоритм. Это связано с тем, что экономия времени за счёт распараллеливания вычислений значительно меньше дополнительных временных затрат на создание потоков или процессов
2. Ознакомился с простейшими принципами параллельных вычислений на C++ и соответствующими библиотеками